SPENDWISE AI

AI-POWERED PERSONAL EXPENSE TRACKING
& FINANCIAL INTELLIGENCE PLATFORM

PRODUCT REQUIREMENTS DOCUMENT

Version 1.0 | November 2026 | Web Application

Track Smarter. Spend Better. Save More.

Document purpose	Define the product scope, requirements, user flows, technical architecture and acceptance criteria for the SpendWise AI prototype.
Target build time	3-5 hours for a functional academic prototype, subject to integration and testing.
Primary platform	Responsive web application for desktop, tablet and mobile browsers.

Document Contents

Section	Contents
01	Product Overview
02	Product Objectives & Target Users
03	Scope and Prioritization
04	Functional Requirements
05	User Flow
06	Technical Architecture
07	Data Model
08	UI/UX Requirements
09	Non-Functional Requirements
10	Acceptance Criteria & Testing
11	Delivery Plan
12	Risks, Assumptions & Future Enhancements

01. Product Overview

SpendWise AI is an AI-powered personal finance management platform that helps users record income and expenses, organize transactions, manage category-wise budgets and understand their financial behaviour. It combines transaction processing, data visualization and AI-generated explanations in one interface.

Problem statement

People often have limited visibility into day-to-day spending. Manual tracking can be inconsistent, transaction descriptions are difficult to organize, and users may not notice category-level overspending until the end of the month.

Proposed solution

Provide a centralized web application where users can enter or import transactions, receive category suggestions, monitor budgets, explore spending trends and generate summaries from their own transaction history.

Product positioning

A student-friendly personal finance dashboard demonstrating full-stack development, NLP-based transaction categorization and AI-assisted financial analytics.

02. Product Objectives & Target Users

Objectives

- Simplify personal income and expense tracking.
- Suggest transaction categories using AI and fallback rules.
- Present financial data through clear dashboards and charts.

- Help users set and monitor monthly category budgets.
- Highlight spending patterns and unusual transactions.
- Generate summaries grounded in the user's recorded data.

Target users

User segment	Primary need
College students	Track daily expenses and stay within a limited budget.
Working professionals	Understand monthly income, expenses and savings.
Freelancers	Monitor variable income and spending.
General users	Build awareness of spending patterns.

03. Scope and Prioritization

In scope - Version 1.0

- User registration, login and secure user-specific access.
- Dashboard with income, expenses, balance and savings rate.
- Transaction create, read, update and delete (CRUD).
- AI-assisted transaction categorization with manual correction.
- CSV import and export.
- Category-wise monthly budgets and utilization indicators.
- Monthly and category-wise analytics.
- AI-generated summaries and spending observations.

Out of scope - Version 1.0

- Direct bank account connectivity or automatic bank synchronization.
- UPI payment initiation or money transfers.
- Investment execution, credit scoring or lending decisions.
- Storage of bank passwords, OTPs or payment credentials.

Priority	Features	Release expectation
P0 - Must have	Authentication, dashboard, transaction CRUD, database	Core prototype
P1 - Should have	Categorization, budgets, analytics	Demonstration-ready
P2 - Nice to have	AI insights, CSV import/export, advanced reports	If time permits

04. Functional Requirements

FR-01 User Authentication

Users can sign up, log in, log out and reset their password. Protected pages must require a valid session. All financial records must be associated with the authenticated user.

Acceptance criteria: A registered user can sign in; unauthenticated users cannot access protected pages; one user cannot read another user's records.

FR-02 Financial Dashboard

Display total income, total expenses, available balance, savings rate, monthly spending trends, category distribution and recent transactions. All values must be calculated from stored transactions and respect the selected date range.

Core calculations

Balance = Total income - Total expenses
Savings rate = (Income - Expenses) / Income × 100
If income is zero, show savings rate as unavailable rather than dividing by zero.

FR-03 Transaction Management

Users can add, view, edit, delete, search, filter and sort transactions. Required fields are transaction name, positive amount, transaction type, category and date. Description is optional.

Field	Type	Requirement
Transaction ID	UUID	System-generated
Transaction name	Text	Required
Amount	Numeric	Required; greater than zero
Type	Income / Expense	Required
Category	Predefined category	Required
Date	Date	Required
Description	Text	Optional

FR-04 AI Transaction Categorization

The system suggests a category based on transaction narration or description. Suggested categories include Food, Shopping, Travel, Entertainment, Bills, Healthcare, Education, Rent, Salary, Investments and Others. Users can override suggestions. When the AI service is unavailable or uncertain, the system must use a rule-based fallback or request manual selection.

Example mapping: “SWIGGY ORDER” → Food; “UBER TRIP” → Travel; “NETFLIX SUBSCRIPTION” → Entertainment; “MONTHLY SALARY” → Salary.

FR-05 Budget Management

Users can create and edit monthly limits for spending categories. The application displays actual spending, remaining budget and utilization percentage. Show a warning when spending reaches or exceeds the configured limit.

Budget utilization = Actual category spending / Category budget × 100.

FR-06 Financial Analytics

Provide monthly income-versus-expense charts, category-wise expense distribution, savings trends, daily spending patterns and budget utilization. Filters must update the charts using the same date range and transaction dataset.

FR-07 AI Financial Insights

Generate summaries from actual transaction records. Insights may identify top spending categories, month-over-month changes, unusual transaction amounts and budget pressure. Every numerical statement must be traceable to the underlying data. Label outputs as AI-generated suggestions, not professional financial advice.

FR-08 CSV Import and Export

Import CSV files with column mapping, validation, duplicate checks and a confirmation step before saving. Export all or filtered transactions as CSV. Invalid rows must be reported clearly and must not be silently imported.

05. User Flow

Step	User action	System response
1	Open SpendWise AI	Landing or login page
2	Sign up / log in	Authenticated session
3	View dashboard	Financial overview
4	Add transaction or import CSV	Transaction records captured
5	Categorize transaction	AI suggestion or fallback rule
6	Review budgets and analytics	Spending and budget status
7	Generate AI insights / export report	Summary and downloadable data

06. Technical Architecture

The proposed architecture uses a React client, Supabase for authentication and PostgreSQL storage, and a secure server-side function for Gemini API calls. The browser must not contain a Gemini secret key.

Layer	Technology	Responsibility
Frontend	React + Vite	Pages, forms, navigation and client-side state.
Styling	Tailwind CSS	Responsive layout and design system.
Authentication / Backend	Supabase	Authentication, API access and server functions.
Database	PostgreSQL	Users, transactions, budgets and insights.
AI service	Gemini API via server-side function	Category suggestions and grounded summaries.
Charts	Recharts	Financial analytics and data visualization.
Deployment	Vercel	Web application hosting.

AI processing sequence

- Receive transaction narration and permitted contextual fields.
- Normalize text and remove irrelevant formatting.
- Call the AI service with a constrained category list and structured-output expectation.
- Validate the returned category and confidence information.
- Use a rule-based fallback or manual review when the response is invalid or uncertain.
- Save the user-confirmed category and use stored records for analytics.

Security controls

- Keep API secrets in server-side environment variables.
- Enable Supabase Row Level Security on user-owned tables.

- Validate all inputs and uploaded CSV files.
- Limit access to each user's own records.
- Avoid storing bank credentials, OTPs or unnecessary sensitive information.

07. Data Model

Table	Key fields	Purpose
profiles	id, name, email, created_at	User profile; id linked to authentication user.
transactions	id, user_id, name, amount, type, category, date, description	Income and expense records.
budgets	id, user_id, category, monthly_limit, month	Monthly category limits.
ai_insights	id, user_id, insight, generated_at	Generated summaries and observations.

Relationships: one user can own many transactions, budgets and AI insights. Use foreign keys to the authenticated user and enforce ownership through Row Level Security policies. Store monetary values using a fixed-precision numeric type rather than floating-point.

08. UI/UX Requirements

Page	Required elements
Login / Signup	Email and password forms, validation, reset-password flow.
Dashboard	Metric cards, date filter, charts, recent transactions, AI insight panel.
Transactions	Transaction table, add/edit form, search, filters and CSV actions.
Budgets	Category budget cards, progress bars, remaining amount and warnings.
Analytics	Monthly trends, category breakdown, income-versus-expense chart.
AI Assistant	Generated summary, supporting metrics, limitations and refresh action.

Visual direction

- Premium fintech style using navy, white and teal accents.
- Consistent spacing, readable typography and clear hierarchy.
- Responsive sidebar or mobile navigation.
- Accessible contrast, visible form labels and keyboard-friendly controls.
- Use loading, empty, success and error states; avoid decorative controls that do not work.

09. Non-Functional Requirements

Area	Requirement
Performance	Common dashboard views should load promptly for typical student-project datasets.
Security	Authentication, server-side secret management and database access policies.

Area	Requirement
Privacy	Collect only necessary data and isolate each user's financial records.
Reliability	Handle AI/API failure without losing transactions; provide deterministic fallback.
Usability	Clear labels, simple navigation and understandable financial summaries.
Compatibility	Support current desktop and mobile browsers.
Data integrity	Validate amounts, dates, types, ownership and imported rows.

10. Acceptance Criteria & Testing

Test case	Expected result
Register a new user	Account is created and user can sign in.
Add an expense	Transaction appears and expense total updates.
Add income	Income total and balance update correctly.
Edit or delete a transaction	All dependent metrics and charts recalculate.
Categorize a narration	Valid category suggestion is displayed; user can override it.
AI service unavailable	Fallback logic or manual selection remains available.
Exceed a budget	Budget warning and utilization update.
Upload malformed CSV	Invalid rows are identified; no silent partial import.
Sign in as a different user	The user cannot access another user's data.
Generate monthly insights	Displayed figures match the selected period's stored records.

11. Delivery Plan

Target implementation time: approximately 4 hours 15 minutes within the requested 3-5 hour window. Actual duration depends on developer experience, API setup and debugging.

Phase	Task	Time
1	Project setup, navigation and dashboard UI	45 min
2	Supabase authentication and database setup	45 min
3	Transaction CRUD and validation	45 min
4	Charts, analytics and budgets	45 min
5	AI categorization and insight generation	45 min
6	Testing, fixes and deployment	30 min
	TOTAL ESTIMATED TIME	4 hr 15 min

12. Risks, Assumptions & Future Enhancements

Risks and mitigations

Risk	Mitigation
AI output may be inaccurate	Restrict output categories, validate results and allow manual correction.
AI API rate limits or outage	Use cached results where appropriate and rule-based fallback.
CSV formats vary	Provide column mapping, validation and preview before import.
Incorrect financial calculations	Use fixed-precision monetary values and test edge cases.
Data exposure	Enable RLS, validate ownership and keep secrets server-side.

Assumptions

- The first release is a prototype for demonstration and academic evaluation.
- Users manually enter transactions or import supported CSV files.
- The AI provider is accessed through a secure server-side function.
- Sample data may be provided for demonstration but must be clearly labelled as demo data.

Future enhancements

- OCR-based receipt scanning.
- Voice-based expense entry.
- Predictive monthly expense forecasting.
- Recurring transaction detection.
- Multi-currency support.
- Optional bank statement integrations, subject to consent and provider availability.

Product definition
SpendWise AI is an intelligent financial management platform combining transaction processing, NLP-based expense categorization, financial analytics and AI-assisted insights to help individuals make informed spending decisions.